

Research Notes on Hierarchical clustering

Hierarchical clustering

>> Section 1

$$d_H(A, B) = \max \{ d_{\sim}(A, B), d_{\sim}(B, A) \}$$
$$\{\displaystyle d_H(A,B)=\max\{\tilde{d}(A,B),\tilde{d}(B,A)\}$$

>> Section 2

Agglomerative: Agglomerative clustering, often referred to as a "bottom-up" approach, begins with each data point as an individual cluster. At each step, the algorithm merges the two most similar clusters based on a chosen distance metric (e.g., Euclidean distance) and linkage criterion (e.g., single-linkage, complete-linkage). This process continues until all data points are combined into a single cluster or a stopping criterion is met. Agglomerative methods are more commonly used due to their simplicity and computational efficiency for small to medium-sized datasets. **Divisive:** Divisive clustering, known as a "top-down" approach, starts with all data points in a single cluster and recursively splits the cluster into smaller ones. At each step, the algorithm selects a cluster and divides it into two or more subsets, often using a criterion such as maximizing the distance between resulting clusters. Divisive methods are less common but can be useful when the goal is to identify large, distinct clusters first. In general, the merges and splits are determined in a greedy manner. The results of hierarchical clustering are usually presented in a dendrogram. Hierarchical clustering has the distinct advantage that any valid measure of distance can be used. In fact, the observations themselves are not required: all that is used is a matrix of distances. On the other hand, except for the special case of single-linkage distance, none of the algorithms (except exhaustive search in $O(2^n)$) can be guaranteed to find the optimum solution.

>> Section 3

Overview HAL-x is a hierarchical density clustering algorithm introduced for large, high-dimensional point clouds. Rather than agglomerating solely from a fixed linkage criterion on a precomputed distance matrix, HAL-x builds an initial set of high-density "pure" clusters in a low-dimensional embedding of a downsampled subset of the data, then merges clusters using supervised linkage: it trains supervised classifiers on the original (unreduced) feature space to measure how reliably two proposed clusters can be separated, and uses those separation scores to decide merge order. The method outputs a hierarchy that can be read as a tree of merges together with a predictive model that can assign labels to held-out or newly collected cells without re-running the entire clustering pipeline on every point during exploratory iterations. In outline, the published implementation

proceeds by (i) normalizing and downsampling for dimensionality reduction (commonly t-SNE or UMAP), (ii) estimating density in the embedded space to obtain initial clusters, (iii) constructing a sparse k-nearest neighbor graph between clusters with edge weights derived from classifier-based separability on unreduced coordinates, and (iv) agglomeratively merging the weakest edges while retaining cross-validated accuracy information so users can obtain multiple resolutions from one fitted model.

>> Section 4

Commercial implementations MATLAB includes hierarchical cluster analysis. SAS includes hierarchical cluster analysis in PROC CLUSTER. Mathematica includes a Hierarchical Clustering Package. NCSS includes hierarchical cluster analysis. SPSS includes hierarchical cluster analysis. Qlucore Omics Explorer includes hierarchical cluster analysis. Stata includes hierarchical cluster analysis. CrimeStat includes a nearest neighbor hierarchical cluster algorithm with a graphical output for a Geographic Information System.

>> Section 5

In this visualization, each point begins as its own cluster. At each step, the two clusters with the smallest distance between them are merged. On the left, dashed lines indicate which clusters are combined, and a running list of cluster groupings shows how clusters evolve over time. On the right, the dendrogram is constructed simultaneously, with each merge represented by a new branch. The height of each merge in the dendrogram corresponds to the distance or dissimilarity between the clusters being combined. Merges that occur at lower heights represent clusters that are more similar, while merges at higher heights indicate greater dissimilarity. Color-coding is used to associate each merge step in the data visualization with its corresponding branch in the dendrogram.

>> Section 6

ALGLIB implements several hierarchical clustering algorithms (single-link, complete-link, Ward) in C++ and C# with $O(n^2)$ memory and $O(n^3)$ run time. ELKI includes multiple hierarchical clustering algorithms, various linkage strategies and also includes the efficient SLINK, CLINK and Anderberg algorithms, flexible cluster extraction from dendrograms and various other cluster analysis algorithms. Julia has an implementation inside the Clustering.jl package. Octave, the GNU analog to MATLAB implements hierarchical clustering in function "linkage". Orange, a data mining software suite, includes hierarchical clustering with interactive dendrogram visualisation. R has built-in functions and packages that provide functions for hierarchical clustering. SciPy implements hierarchical

Research Notes on Hierarchical clustering

Hierarchical clustering

clustering in Python, including the efficient SLINK algorithm. scikit-learn also implements hierarchical clustering in Python. Weka includes hierarchical cluster analysis.

>> Section 7

In practice, the selection of the number of clusters may also depend on domain knowledge or application-specific requirements. Quantitative measures such as the silhouette coefficient can be used to evaluate the quality of different clusterings. Because hierarchical clustering is sensitive to the choice of distance metric and linkage criterion, the optimal number of clusters may vary depending on these factors. This method builds the hierarchy from the individual elements by progressively merging clusters. In our example, we have six elements $\{a\}$ $\{b\}$ $\{c\}$ $\{d\}$ $\{e\}$ and $\{f\}$. The first step is to determine which elements to merge in a cluster. Usually, we want to take the two closest elements, according to the chosen distance. Optionally, one can also construct a distance matrix at this stage, where the number in the i -th row j -th column is the distance between the i -th and j -th elements. Then, as clustering progresses, rows and columns are merged as the clusters are merged and the distances updated. This is a common way to implement this type of clustering, and has the benefit of caching distances between clusters. A simple agglomerative clustering algorithm is described in the single-linkage clustering page; it can easily be adapted to different types of linkage (see below). Suppose we have merged the two closest elements b and c , we now have the following clusters $\{a\}$, $\{b, c\}$, $\{d\}$, $\{e\}$ and $\{f\}$, and want to merge them further. To do that, we need to take the distance between $\{a\}$ and $\{b, c\}$, and therefore define the distance between two clusters. Usually the distance between two clusters A $\{\displaystyle \mathcal{A}\}$ and B $\{\displaystyle \mathcal{B}\}$ is one of the following: